Projeto Luanny - Implementação

Documento Base de Implementação

Coletor de Dados de Acessibilidade no Instagram para Pesquisa Acadêmica

1. Resumo Executivo

O documento original “Projeto Luanny.pdf” descreve uma ideia promissora: construir um coletor automatizado para analisar recursos de acessibilidade presentes em postagens de perfis do Instagram, com foco em uso acadêmico e, possivelmente, em perfis de órgãos públicos. A proposta geral é válida, mas o material original mistura visão de produto, pseudocódigo e trechos de implementação ainda incompletos. Por isso, ele não deve ser usado como especificação final de desenvolvimento.

Este documento substitui aquele plano por uma versão mais completa e operacional. O objetivo aqui é servir como base de trabalho para um desenvolvedor ou equipe terceirizada, cobrindo:

- o que será construído;
- o que está bem definido;
- o que ainda está indefinido;
- os principais riscos técnicos;
- a forma recomendada de implementação;
- as decisões que precisam ser tomadas antes do início da codificação;
- o plano por fases, da concepção até a entrada em produção.

A intenção é que um agente humano com experiência geral em programação, mesmo sem grande familiaridade com Python, consiga seguir este documento e conduzir o projeto até uma versão funcional.

---

2. Objetivo do Produto

Construir uma aplicação capaz de coletar, estruturar e armazenar informações de acessibilidade de publicações públicas do Instagram, com foco em análise acadêmica e auditoria de práticas de acessibilidade digital.

O sistema deve:

- acessar perfis públicos do Instagram;
- identificar postagens a partir do perfil-alvo;
- abrir cada postagem individualmente;
- extrair metadados relevantes de acessibilidade;
- gerar saídas estruturadas para análise posterior;
- preservar evidências técnicas da coleta;
- permitir repetição controlada do processo.

---

3. Problema que o Produto Resolve

Pesquisadores e avaliadores de comunicação digital precisam analisar como perfis institucionais usam recursos de acessibilidade em redes sociais. Isso inclui, por exemplo:

- texto alternativo de imagens;
- pistas de descrição automática do Instagram;
- legendas em vídeos;
- marcas textuais de audiodescrição na legenda;
- uso de ARIA labels e outros atributos de acessibilidade visíveis no DOM.

Hoje, essa análise costuma ser manual, lenta e pouco reprodutível. O produto pretende automatizar essa coleta com trilha de auditoria suficiente para suportar pesquisa acadêmica.

---

4. Escopo do Produto

4.1 Escopo incluído na primeira versão funcional

A primeira versão funcional deve contemplar:

- autenticação no Instagram com conta operacional dedicada;
- reaproveitamento de sessão autenticada;
- navegação até um perfil público-alvo;
- descoberta de URLs/IDs de postagens;
- abertura individual de cada postagem;
- extração de dados de acessibilidade e contexto;
- exportação em JSON e CSV;
- geração de evidências técnicas mínimas;
- logs de execução;

- tratamento básico de falhas;
- testes unitários e de integração.

4.2 Escopo fora da primeira versão funcional

Os itens abaixo devem ser tratados como fases posteriores, não como requisito obrigatório para o primeiro release:

- download e transcrição de áudio com Whisper;
- classificação avançada de audiodescrição por NLP;
- painel web interativo;
- banco de dados relacional em produção;
- orquestração distribuída;
- scraping massivo em larga escala;
- suporte multilíngue sofisticado;
- coleta de stories, lives ou DMs;
- integração com APIs de terceiros não essenciais.

---

5. O que o Documento Original Tem de Sólido

Apesar dos problemas, o PDF original acerta em alguns pontos importantes:

5.1 Boa decomposição de responsabilidades

Ele separa o sistema em blocos conceitualmente corretos:

- autenticação;
- extração de acessibilidade;
- rolagem/navegação;
- processamento de áudio;
- orquestração principal;
- integridade acadêmica.

Essa separação deve ser preservada, com ajustes.

5.2 Direção tecnológica plausível

A escolha de Python com Playwright é adequada porque:

- Playwright lida bem com sites dinâmicos;

- Python facilita parsing, exportação e análise;
- a combinação atende bem a protótipos robustos de scraping com browser real.

5.3 Preocupação com integridade da pesquisa

A ideia de gerar screenshot, HTML e hash de verificação é excelente e deve ser mantida.

5.4 Saídas úteis para pesquisa

JSON detalhado e CSV resumido são formatos corretos para reuso acadêmico.

---

6. O que Ainda Não Está Pronto no Documento Original

O PDF original não é suficiente para implementação direta. Os principais problemas são estes.

6.1 Código incompleto ou inconsistente

Há trechos com:

- imports ausentes;
- indentação quebrada;
- variáveis não definidas;
- strings corrompidas pela conversão do PDF;
- dependências usadas mas não declaradas;
- campos esperados em um módulo mas nunca produzidos em outro.

6.2 Modelo de dados não está fechado

O documento não define de forma rígida:

- quais campos cada post deve conter;
- como representar mídia múltipla;
- como representar erros parciais;
- como versionar o schema;
- como registrar evidências.

6.3 Estratégia de coleta pouco confiável

O plano original tenta extrair muita coisa diretamente da listagem do perfil. Isso é arriscado. O ideal é:

1. descobrir as postagens no grid do perfil;

2. abrir cada postagem em sua própria página;
3. extrair os dados em contexto estável.

6.4 Dependências e requisitos faltantes

Por exemplo:

- `requests` é usado, mas não aparece em `requirements.txt`;
- o módulo de áudio depende de download de vídeo, mas o fluxo para obter URL real do vídeo não está definido;
- o uso de Whisper não considera custo operacional, tempo e erros.

6.5 Seletores frágeis

O documento assume seletores e labels como se fossem estáveis, por exemplo:

- elementos com `aria-label="Home"`;
- textos específicos como “Image may contain:”.

Isso pode quebrar por:

- mudança de interface do Instagram;
- localização do idioma;
- testes A/B;
- diferenças entre web desktop e outras variantes.

6.6 Lacunas de compliance e operação

O PDF menciona ética, mas não especifica:

- limites de coleta por execução;
- política de retry;
- política de backoff;
- critérios para interrupção automática;
- estratégia de rotação de conta operacional;
- retenção e proteção de credenciais.

---

7. Produto Proposto: Versão Corrigida e Implementável

7.1 Descrição funcional

O sistema será uma ferramenta de linha de comando, com execução local ou em servidor, capaz de receber um nome de perfil do Instagram e produzir uma coleta estruturada dos recursos de acessibilidade observáveis nas postagens públicas desse perfil.

Entrada principal

- nome do perfil do Instagram;
- quantidade máxima de posts a coletar;
- diretório de saída;
- modo headless ou com navegador visível;
- opção para capturar evidências;
- opção para ativar análise de áudio em fase posterior.

Saídas principais

- arquivo JSON com todos os dados detalhados;
- arquivo CSV com resumo analítico;
- diretório de evidências por post;
- log completo da execução;
- relatório de execução.

---

8. Arquitetura Recomendada

8.1 Visão em alto nível

A arquitetura recomendada é modular.

Módulos principais

- `config` : leitura e validação de configuração;
- `auth` : login e reaproveitamento de sessão;
- `browser` : inicialização do Playwright e contexto do navegador;
- `profile_discovery` : descoberta de posts no perfil;
- `post_extractor` : extração de dados detalhados do post;
- `evidence_capture` : screenshot, HTML e hash;
- `exporters` : JSON, CSV e relatório;
- `audio_analysis` : fase futura;
- `cli` : comandos de execução;
- `logging` : logs estruturados;

- `models` : schema de dados.

8.2 Fluxo principal

1. Carregar configuração.
2. Inicializar navegador e contexto.
3. Reusar sessão autenticada, se existir.
4. Autenticar, se necessário.
5. Abrir perfil-alvo.
6. Descobrir postagens até o limite definido.
7. Para cada postagem:
   - abrir a página do post;
   - extrair dados;
   - capturar evidências, se habilitado;
   - registrar erros sem abortar toda a execução.
8. Consolidar resultados.
9. Exportar arquivos.
10. Encerrar navegador e gerar relatório final.

---

9. Modelo de Dados Recomendado

Cada postagem deve gerar um registro com schema estável.

9.1 Registro de post

Campos mínimos:

- `post_id`
- `post_url`
- `profile_username`
- `scraped_at`
- `caption`
- `hashtags`
- `mentions`
- `post_type`
- `is_video`
- `is_carousel`
- `accessibility_data`

- `media`
- `aria_labels`
- `has_audio_description_textual_hint`
- `video_analysis`
- `evidence`
- `errors`

9.2 Campo `media`

Lista de objetos, cada um com:

- `media_index`
- `media_type`
- `src`
- `alt_text`
- `alt_text_source`
- `is_auto_generated_alt`
- `poster`
- `dimensions` quando disponível

9.3 Campo `aria_labels`

Lista com:

- `tag_name`
- `role`
- `aria_label`
- `context_hint`

9.4 Campo `video_analysis`

Na v1 pode ser simples:

- `has_captions_visual_hint`
- `has_audio_description_textual_hint`

Na fase de áudio, acrescentar:

- `transcript`
- `ad_likelihood`
- `keywords_found`
- `audio_processing_error`

9.5 Campo `evidence`

- `screenshot_path`
- `html_path`
- `content_hash`
- `captured_at`

9.6 Campo `errors`

Lista de erros não fatais relacionados ao post.

---

10. Estratégia de Implementação Recomendada

10.1 Estratégia correta de scraping

A coleta não deve depender exclusivamente do feed infinito como fonte de dados completos. O processo ideal é em duas etapas.

Etapa A: descoberta

No perfil-alvo, identificar os links das postagens até um limite.

Objetivos:

- deduplicar posts;
- armazenar `post_id` e `post_url`;
- evitar depender da renderização completa do feed.

Etapa B: extração detalhada

Abrir cada post individualmente e extrair:

- caption;
- imagens e alt texts;
- presença de vídeos;
- labels acessíveis;
- sinais textuais de audiodescrição;
- metadados adicionais disponíveis no DOM.

Essa abordagem reduz fragilidade e melhora a repetibilidade.

---

11. Riscos Técnicos Principais

11.1 Risco de autenticação

Instagram pode:

- exigir login;
- apresentar checkpoint;
- disparar 2FA;
- bloquear comportamento suspeito.

Mitigação:

- usar conta operacional dedicada;
- armazenar sessão autenticada;
- limitar volume por execução;
- evitar relogar desnecessariamente.

11.2 Risco de seletor quebrar

Os seletores de DOM do Instagram podem mudar sem aviso.

Mitigação:

- centralizar seletores em um único módulo;
- usar estratégias de fallback;
- registrar HTML de erro;
- manter testes com snapshots/fixures.

11.3 Risco de idioma/localização

Textos como “Image may contain:” podem aparecer em outro idioma.

Mitigação:

- padronizar navegador em idioma conhecido;
- armazenar texto bruto;
- evitar depender de uma única string;
- documentar a limitação.

11.4 Risco de bloqueio por automação

Mitigação:

- usar atrasos humanos controlados;
- limitar taxa de requisições;
- evitar scraping em larga escala inicialmente;
- usar navegador real com Playwright.

11.5 Risco de inconsistência de dados

Se parte dos campos falhar, a coleta não pode quebrar por completo.

Mitigação:

- schema tolerante a campos ausentes;
- logging por post;
- lista de erros por item;
- exportação mesmo com falhas parciais.

11.6 Risco do módulo de áudio

Baixar e transcrever vídeos é caro e frágil.

Mitigação:

- tornar opcional;
- implementar apenas depois da v1 estável;
- separar em pipeline posterior.

---

12. Decisões que Precisam Ser Tomadas Antes da Implementação

Estas decisões devem ser fechadas antes do desenvolvimento principal.

12.1 Perfis-alvo

Decidir se o sistema será restrito a:

- perfis públicos apenas;
- perfis institucionais apenas;
- qualquer perfil público autorizado pelo operador.

Recomendação: perfis públicos apenas.

12.2 Escopo de conteúdo

Decidir se a v1 cobre:

- apenas posts do feed;
- posts e reels;
- carrosséis;
- vídeos.

Recomendação: posts do feed, incluindo imagem única, carrossel e vídeo, desde que apareçam como postagens públicas convencionais.

12.3 Volume por execução

Definir limite operacional padrão.

Recomendação:

- padrão de 30 a 50 posts por execução;
- máximo configurável com cautela.

12.4 Evidência obrigatória

Decidir se todo post gera evidência ou apenas amostras.

Recomendação:

- evidência obrigatória na v1 para todos os posts coletados, se o volume for moderado;
- para volumes maiores, permitir modo amostral.

12.5 Audiodescrição

Decidir se análise de áudio é parte da primeira entrega.

Recomendação:

- não;
- manter somente heurística textual na v1;
- Whisper entra na v1.1 ou v2.

12.6 Banco de dados

Decidir se a primeira versão persistirá em arquivos ou banco.

Recomendação:

- arquivos JSON/CSV na v1;

- banco só se houver demanda operacional real.

12.7 Ambiente de execução

Decidir entre:

- execução local;
- servidor único;
- container.

Recomendação:

- desenvolver localmente;
- publicar em container para produção.

---

13. Plano por Fases do Desenvolvimento

Fase 0. Concepção e Validação de Escopo

Objetivo

Transformar a ideia em especificação executável.

Entregáveis

- documento de requisitos;
- decisões de escopo fechadas;
- schema preliminar do dado;
- critérios de aceite por fase;
- registro de riscos.

Critérios de aceite

- escopo v1 definido;
- requisitos fora da v1 documentados;
- conta operacional e política de uso aprovadas.

Fase 1. Estrutura do Projeto e Ambiente

Objetivo

Criar base de desenvolvimento reproduzível.

Atividades

- criar repositório;
- definir estrutura de pastas;
- configurar ambiente virtual;
- instalar Playwright e Chromium;
- configurar `.env.example`;
- configurar lint, testes e logging.

Estrutura sugerida

- `src/luanny/`
- `tests/`
- `data/`
- `artifacts/`
- `logs/`
- `scripts/`
- `docs/`

Entregáveis

- projeto executável localmente;
- comando de instalação documentado;
- comando de teste documentado.

Critérios de aceite

- um novo desenvolvedor consegue clonar e subir o ambiente em até 30 minutos.

Fase 2. Configuração e Modelos de Dados

Objetivo

Padronizar leitura de configuração e schema.

Atividades

- criar classes de configuração;

- validar variáveis de ambiente;
- definir modelos com Pydantic ou dataclasses;
- definir schema do JSON final.

Entregáveis

- módulo `config`;
- módulo `models`;
- validação de parâmetros de entrada.

Critérios de aceite

- execução falha cedo e de forma clara quando faltam credenciais ou parâmetros.

Fase 3. Autenticação e Sessão

Objetivo

Garantir acesso confiável e reutilizável ao Instagram.

Atividades

- implementar login;
- tratar sessão persistida;
- salvar `storage_state`;
- reautenticar apenas quando necessário;
- preparar suporte básico a 2FA manual.

Entregáveis

- módulo `auth`;
- arquivo de estado de sessão;
- logs de autenticação.

Critérios de aceite

- duas execuções consecutivas não exigem novo login quando a sessão ainda é válida.

Fase 4. Descoberta de Postagens

Objetivo

Listar posts do perfil-alvo de forma estável.

Atividades

- abrir perfil;
- rolar com limites controlados;
- capturar links `/p/` ou equivalentes válidos;
- deduplicar por `post_id`;
- respeitar limite máximo.

Entregáveis

- módulo `profile_discovery`;
- lista de posts com ID e URL.

Critérios de aceite

- para um perfil de teste, o sistema encontra N posts distintos e válidos sem duplicação.

Fase 5. Extração de Dados do Post

Objetivo

Extrair o máximo de dados relevantes por postagem.

Atividades

- abrir cada post em página própria;
- extrair caption;
- extrair hashtags e menções;
- identificar imagem, vídeo e carrossel;
- extrair alt text;
- extrair aria-labels;
- capturar sinais textuais de acessibilidade.

Observação

A extração deve aceitar ausência de campos. Nem todo post terá todos os elementos.

Entregáveis

- módulo `post_extractor`;
- registro estruturado por post.

Critérios de aceite

- pelo menos um conjunto de posts de teste retorna dados coerentes e schema válido.

Fase 6. Evidência e Integridade Acadêmica

Objetivo

Garantir rastreabilidade da coleta.

Atividades

- salvar screenshot por post;
- salvar HTML bruto por post;
- gerar hash SHA-256;
- registrar timestamp de captura.

Entregáveis

- módulo `evidence_capture`;
- diretórios organizados por execução e por post.

Critérios de aceite

- cada post coletado possui conjunto mínimo de evidências acessível.

Fase 7. Exportação e Relatórios

Objetivo

Gerar saídas utilizáveis por pesquisadores.

Atividades

- exportar JSON detalhado;

- exportar CSV resumo;
- gerar relatório de execução;
- registrar contagem de erros, sucessos e falhas parciais.

Entregáveis

- módulo `exporters`;
- convenção de nomes dos arquivos.

Critérios de aceite

- pesquisador consegue abrir CSV e JSON sem tratamento adicional complexo.

Fase 8. Tratamento de Erros, Robustez e Observabilidade

Objetivo

Tornar o sistema operacionalmente confiável.

Atividades

- retries com backoff;
- timeouts configuráveis;
- logs estruturados;
- tratamento de falhas por post;
- captura de evidência especial quando ocorrer erro.

Entregáveis

- política de erro documentada;
- logs legíveis;
- execução tolerante a falhas parciais.

Critérios de aceite

- uma falha em um post não aborta toda a coleta, salvo erro fatal de autenticação ou bloqueio geral.

Fase 9. Testes

Objetivo

Proteger o projeto contra regressões.

Tipos de teste

- unitários;
- integração com fixtures HTML;
- smoke test E2E em perfil de teste.

Casos mínimos

- parser de hashtags;
- parser de menções;
- normalização de registro;
- geração de hash;
- exportação CSV;
- leitura de configuração;
- reaproveitamento de sessão;
- extração em páginas salvas.

Critérios de aceite

- suíte principal verde em CI;
- smoke test executável sob demanda.

Fase 10. Módulo de Áudio Opcional

Objetivo

Adicionar capacidade de inferir audiodescrição em vídeos.

Observação

Esta fase não é obrigatória para a primeira versão funcional.

Atividades

- identificar URL utilizável de mídia de vídeo;
- baixar vídeo temporariamente;
- transcrever com Whisper;

- procurar padrões de descrição;
- anexar resultado ao post.

Riscos

- custo computacional;
- fragilidade;
- direitos e limites operacionais;
- lentidão.

Critérios de aceite

- módulo é opcional, desacoplado e não afeta a coleta principal quando desligado.

Fase 11. Empacotamento e Deployment

Objetivo

Levar o sistema a um ambiente estável de execução.

Estratégia recomendada

Publicar como aplicação containerizada em um servidor simples.

Componentes

- Dockerfile;
- arquivo de variáveis de ambiente;
- volume para dados e evidências;
- agendador;
- rotina de limpeza e retenção.

Recomendação operacional

- um container para execução sob demanda ou agendada;
- persistência em volume;
- credenciais via secret manager ou variáveis seguras;
- logs exportados para arquivo ou agregador.

Critérios de aceite

- uma execução agendada produz artefatos corretos sem intervenção manual.

---

14. Forma Recomendada de Implementação

14.1 Linguagem e stack

Recomendação principal:

- Python 3.11+
- Playwright
- Pydantic
- Typer para CLI
- pytest
- pandas
- python-dotenv
- structlog ou logging padrão
- tenacity para retry

14.2 Organização interna

Recomendação simples e robusta:

- aplicação em pacote Python;
- CLI única como ponto de entrada;
- módulos pequenos e especializados;
- configurações centralizadas.

14.3 Padrão de execução

Exemplo de fluxo esperado:

```
python -m luanny scrape --profile nasa --max-posts 30 --capture-evidence
```

14.4 Política de logs

Cada execução deve registrar:

- início e fim;
- perfil-alvo;
- quantidade de posts solicitada;
- quantidade de posts descobertos;

- quantidade processada com sucesso;
- quantidade com falha parcial;
- erros fatais.

---

15. Critérios de Qualidade da Primeira Versão Funcional

A v1 será considerada pronta quando cumprir todos os itens abaixo:

- autentica com estabilidade razoável;
- reutiliza sessão previamente salva;
- coleta posts públicos de um perfil-alvo;
- extrai dados essenciais de acessibilidade;
- gera JSON e CSV válidos;
- salva evidência mínima;
- lida com falhas parciais;
- possui testes automatizados mínimos;
- possui documentação de instalação e uso;
- pode rodar de forma previsível em ambiente local e containerizado.

---

16. Próximos Passos Mais Recomendados

Passo 1

Fechar as decisões pendentes de escopo, especialmente:

- apenas perfis públicos;
- sem análise de áudio na v1;
- coleta limitada por execução;
- evidência obrigatória.

Passo 2

Criar o repositório e a estrutura-base do projeto.

Passo 3

Implementar `config`, `models`, `cli` e logging antes do scraping.

Passo 4

Implementar autenticação com persistência de sessão.

Passo 5

Implementar descoberta de postagens.

Passo 6

Implementar extração detalhada por post.

Passo 7

Implementar exportação e evidências.

Passo 8

Criar testes e corrigir instabilidades.

Passo 9

Empacotar com Docker e validar execução agendada.

Passo 10

Somente depois considerar o módulo de áudio.

---

17. Conclusão

O documento original tem uma boa intuição de produto, mas não oferece base suficiente para uma implementação segura e previsível. A principal correção conceitual necessária é tratar o projeto como um scraper modular e orientado a evidências, com foco em coleta confiável de posts individuais, não como um conjunto de scripts soltos.

A melhor estratégia é entregar primeiro uma versão funcional enxuta, robusta e auditável, e só depois adicionar recursos pesados como transcrição de vídeo. Com isso, o projeto terá valor acadêmico real, menor risco técnico e caminho claro para manutenção.

Se você quiser, no próximo passo eu posso converter este documento em um **especificação técnica formal pronta para fornecedor**, com:

- requisitos funcionais e não funcionais;
- arquitetura;

- backlog por sprint;
- critérios de aceite por entrega;
- e checklist de homologação final.